UNIVERSIDAD TECMILENIO

ACTIVIDAD: ***Actividad 3***

DOCENTE: ***Francisco Gómez Rubio***

MATERIA: ***Ciberseguridad***

ALUMNO: ***CÉSAR JULIÁN ESPRONCEDA PANTOJA***

MATRICULA: ***AL07040765***

10 de Noviembre del 2026

Índice

Tabla de contenido

Portada 1
Índice 2
Introducción 3
Fase 1 3
Fase 2 6
Fase 3 8
Politica IAM restrictiva en AWS 11
Diagrama de arquitectura 12
Conclusión 13

Introducción

En el presente documento se presenta la Actividad 3 de esta misma materia, la cual contiene 3 fases, las cuáles son: Fase 1 Bucket S3 cifrado y con acceso público bloqueado con Terraform, Fase 2 escaneo de una imagen Docker con Trivy y la fase 3 la cuál es una autentificación JWT en una API probada por OWASP ZAP, al igual se realizó una política IA y un diagrama de la arquitectura, por último es importante mencionar que no use AWS si no LocalStack, por lo que todo el código se encuentra en el GitHub tal como usted Paco nos mencionó, al igual omití información y capturas para no hacer el documento tan largo, tal como las capturas de creación de cuenta en LocalStack y cosas por el estilo, el repositorio es el siguiente:

https://github.com/CesarEsproncedaP/Actividad3-Ciberseguridad-Cesar-Espronceda-AL07040765.git

Fase 1

Con Terraform creé el bucket “**actividad3-seguro-local”** con cifrado AWS-256 y los 4 controles de bloqueo de acceso público. Dicho código “**main.tf”** tiene 3 recursos, debido a que en las versiones más recientes del povider de AWS el cifrado y el bloqueo ya no se declaran dentro del recurso del bucket, el provider apunta al endpoint del LocalStack con credenciales falsas y LocalStack se levanta con Docker Compoose, expuesto solo en 127.0.0.1, todo esto se encuentra en la carpeta “**fase1-terraform”** del repositorio.

LocalStack quedó en estado healthy y terraform plan indicó que se crearían 3 recursos mientras que terraform apply los creó.



```
  }

  # aws_s3_bucket_public_access_block.block will be created
  + resource "aws_s3_bucket_public_access_block" "block" {
      + block_public_acls       = true
      + block_public_policy     = true
      + bucket                  = (known after apply)
      + id                      = (known after apply)
      + ignore_public_acls      = true
      + restrict_public_buckets = true
    }

  # aws_s3_bucket_server_side_encryption_configuration.enc will be created
  + resource "aws_s3_bucket_server_side_encryption_configuration" "enc" {
      + bucket = (known after apply)
      + id     = (known after apply)

      + rule {
          + apply_server_side_encryption_by_default {
              + sse_algorithm     = "AES256"
                # (1 unchanged attribute hidden)
            }
        }
    }

Plan: 3 to add, 0 to change, 0 to destroy.
```

```
  Enter a value: yes

aws_s3_bucket.secure_bucket: Creating...
aws_s3_bucket.secure_bucket: Creation complete after 1s [id=actividad3-seguro-local]
aws_s3_bucket_public_access_block.block: Creating...
aws_s3_bucket_server_side_encryption_configuration.enc: Creating...
aws_s3_bucket_public_access_block.block: Creation complete after 0s [id=actividad3-seguro-local]
aws_s3_bucket_server_side_encryption_configuration.enc: Creation complete after 0s [id=actividad3-seguro-local]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
PS C:\Users\cesar\actividad3-localstack>
PS C:\Users\cesar\actividad3-localstack> terraform state list
aws_s3_bucket.secure_bucket
aws_s3_bucket_public_access_block.block
aws_s3_bucket_server_side_encryption_configuration.enc
PS C:\Users\cesar\actividad3-localstack>
```

Para validar desde la línea de comandos usé AWS CLI con un endpoint local y credenciales ficticias, tal como se ve en la siguiente captura donde el bucket existe, que el cifrado es AES256 y que los 4 bloqueos de acceso público están en true.

```
PS C:\Users\cesar\actividad3-localstack> aws s3 ls --endpoint-url http://localhost:4566
2026-11-09 21:51:09 actividad3-seguro-local
PS C:\Users\cesar\actividad3-localstack> aws s3api get-bucket-encryption --bucket actividad3-seguro-local --endpoint-url http://localhost:4566
{
    "ServerSideEncryptionConfiguration": {
        "Rules": [
            {
                "ApplyServerSideEncryptionByDefault": {
                    "SSEAlgorithm": "AES256"
                },
                "BucketKeyEnabled": false,
                "BlockedEncryptionTypes": {
                    "EncryptionType": [
                        "SSE-C"
                    ]
                }
            }
        ]
    }
}

PS C:\Users\cesar\actividad3-localstack> aws s3api get-public-access-block --bucket actividad3-seguro-local --endpoint-url http://localhost:4566
{
    "PublicAccessBlockConfiguration": {
        "BlockPublicAcls": true,
        "IgnorePublicAcls": true,
        "BlockPublicPolicy": true,
        "RestrictPublicBuckets": true
    }
}

PS C:\Users\cesar\actividad3-localstack>
```

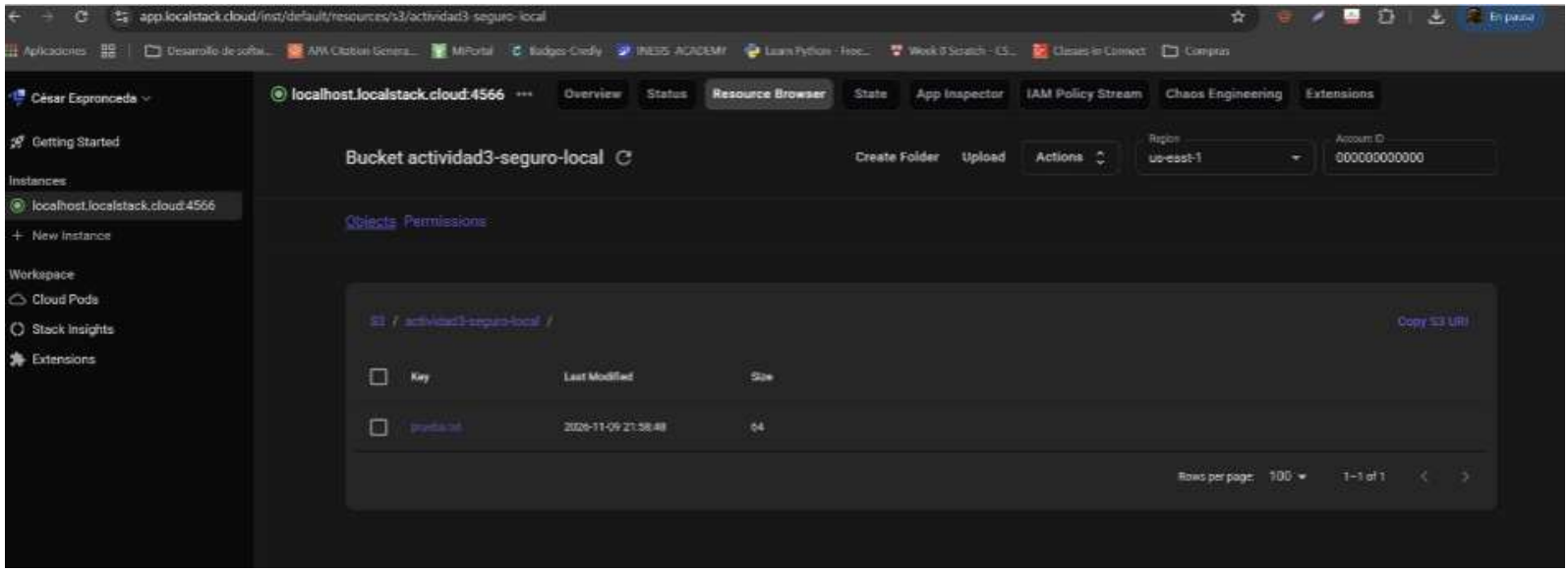
Después subí un archivo de prueba, el cuál intenté descargarlo, el cuál fue permitido por LocalStack pero no se interpretó como que el bucket sea público, ya que es un simulador y no replica las reglas de autorización de AWS, al igual el error de **“NoSuchBucketPolicy”** solo indica que el bucket no tiene una política adjunta, por utlimo, el bucket y el archivo se ven en el Dasboard de Local Stack:

```
PS C:\Users\cesar\actividad3-localstack> "archivo de prueba actividad 3" | Out-File prueba.txt
PS C:\Users\cesar\actividad3-localstack> aws s3 cp prueba.txt s3://actividad3-seguro-local/ --endpoint-url http://localhost:4566
upload: .\prueba.txt to s3://actividad3-seguro-local/prueba.txt
PS C:\Users\cesar\actividad3-localstack> aws s3 ls s3://actividad3-seguro-local/ --endpoint-url http://localhost:4566
2026-11-09 21:58:48         64 prueba.txt
PS C:\Users\cesar\actividad3-localstack>
```

```
PS C:\Users\cesar\actividad3-localstack> aws s3 cp s3://actividad3-seguro-local/prueba.txt - --no-sign-request --endpoint-url http://localhost:4566
◆◆archivo de prueba actividad 3
PS C:\Users\cesar\actividad3-localstack>
```

```
PS C:\Users\cesar\actividad3-localstack> aws s3api get-bucket-policy-status --bucket actividad3-seguro-local --endpoint-url http://localhost:4566

aws: [ERROR]: An error occurred (NoSuchBucketPolicy) when calling the GetBucketPolicyStatus operation: The bucket policy does not exist
PS C:\Users\cesar\actividad3-localstack>
```



Fase 2

Aquí en esta parte construí una imagen con una versión base que estuviera obsoleta, tal como es node 14, la escaneé con Trivy y después la mitigue cambiando la imagen base a node 18 y como esta no fue lo suficiente, a node 22 alpine, escaneando cada cambio, los dockerfile, la aplicación de prueba y los reportes completos de Trivy se encuentran en la carpeta **“fase2-contenedores”** del repositorio. Los escaneos filtran las severidades HIGH y CRITICAL.

Con node 14, Trivy detectó Debian 10.13 y 23 vulnerabilidades críticas y 572 altas en total, además advirtió que esa versión de Debian ya no tiene soporte por lo que en realidad el número podría ser mayor, entre las críticas aparecen paquetes como git, libpython2.7, openssh-client y python2.7.





Con node 18 el resultado empeoró, ya que me arrojaba 118 críticas y 2302 altas, esto se debe a que Trivy detecta con información completa las vulnerabilidades de Debian 12, que es el sistema de esa imagen, mientras que para Debian 10 su detección era insuficiente.



Mientras que con node 22 alpine, la cual es una versión con soporte y una distribución más pequeña, 19 paquetes contra 43 de las imágenes de Debian, el reporte quedo sin vulnerabilidades críticas, tal como se presenta en la siguiente imagen:



Al igual en la siguiente tabla resumo los 3 escaneos relalizados:

Imagen	Sistema base	Paquetes	Críticas	Altas
Node 14	Debian 10.13	413	23	572
Node 18	Debian 12.11	413	228	2302
Node 22 alpine	Alpine 3.24.4	19	0	10

En conclusión paraesta fase, me quedo claro que no solamente actualizando la imagen base reduce las vulnerabilidades, ya que pasar de node 14 a 18 las aumentó, lo que si funciono fue node 22 alpine, que quedo sin críticas.

Fase 3

La API simulada está hecha con Node.js, Express y jsonwebtoken y su código se encuentra en la carpeta **“fase3-api”** del repositorio, y tiene 2 endpoints públicos **“/api/public”** y **“/api/login”** y 3 endpoints protegidos **“/api/perfil”, “/api/admin”** y **“/api/admin/usuarios”**. El login devuelve un token firmado con HS256 que expira en 15 minutos y el secreto no está en el código son que se lee la variable de entorno **“JWT_SECRET”**. La validación se hace con 2 middlewares: **verifyToken** revisa la firma y la expiración, mientras que **requireRole** revisa el rol, ambos se aplican a los endpoints sensibles **“/api/admin”** y **“/api/admin/usuarios”** y **verifyToken** también a **“/api/perfil”**.

Al igual probé los endpoints con curl.exe, sn token y con el token alterado y respondieron con error 401, con el token de un usuario normal, **“/api/admin”** respondió con el error 403 y **“/api/perfil 200”** y con el token del admin los dos endpoints sensibles respondieron con 200.

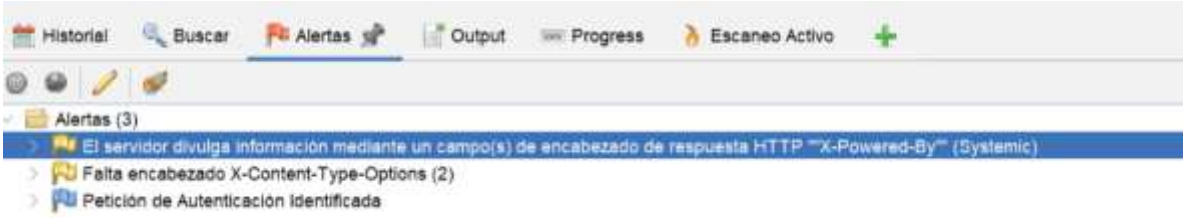






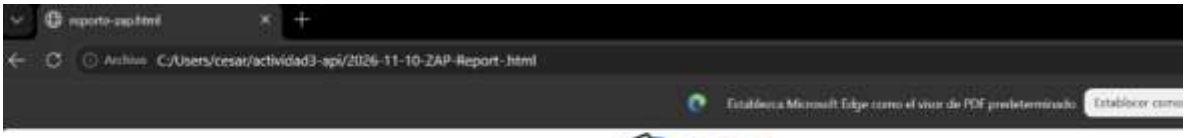
Prueba	Código	Respuesta
Sin token en **/api/admin** y **/apiadmin/usuarios**	401	Token requerido
Token con la firma alterada	401	Token inválido o expirado
Usuario normal en **/api/admin**	403	Permisos insuficientes
Usuario normal en **/api/admin**	200	Perfil del usuario
Admin en **/api/admin** y **/api/admin/usuarios**	200	Acceso permitido

Después importé la definición de la API **"/openapi.json"** en OWASP ZAP y la escaneé, el reporte tuvo 0 alertas altas, 0 medias, 2 bajas y 1 informativa, el servidor divulga que usa Express por el encabezado **X-Powered-By** y la falta el encabezado **X-Content-Type-Options,** Zap no encontró ningún endpoint sensible accesible sin autentificación, en su historial las peticiones sin token a **"/api/perfin"**l **"/api/admin"** y **"/api/admin/usuarios"** respondieron 401.





ID	Fuente	Petición (Tiempo)	Método	URL	Código	Razón
1	Manual	10/11/26, 4:53:00 p.m.	GET	http://127.0.0.1:4000/openapi.json	200	OK
3	Manual	10/11/26, 4:53:00 p.m.	GET	http://127.0.0.1:4000/api/public	200	OK
5	Manual	10/11/26, 4:53:00 p.m.	POST	http://127.0.0.1:4000/api/login	401	Unauthorized
6	Manual	10/11/26, 4:53:00 p.m.	GET	http://127.0.0.1:4000/api/perfil	401	Unauthorized
7	Manual	10/11/26, 4:53:00 p.m.	GET	http://127.0.0.1:4000/api/admin	401	Unauthorized
8	Manual	10/11/26, 4:53:00 p.m.	GET	http://127.0.0.1:4000/api/admin/usuarios	401	Unauthorized



ZAP by Checkmarx **reporte-zap.html**

Sitio: http://127.0.0.1:4000

Generado a mar, 10 nov 2026 17:03:35

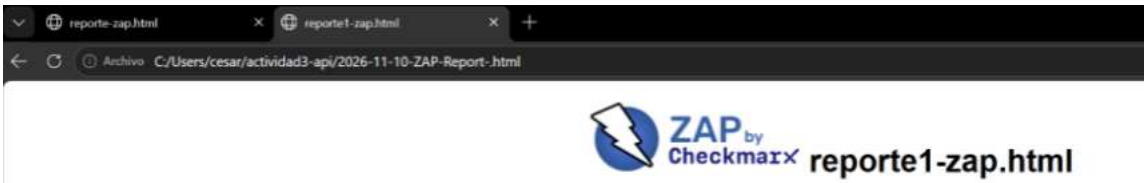
ZAP Versión: 2.17.0

ZAP by Checkmarx

Sumario de Alertas

Nivel de riesgo	Número de Alertas
Alto	0
Medio	0
Bajo	2
Informativo	1

Las 2 alertas bajas se corrigieron agregando el paquete helmet en server.js, que añade encabezados de seguridad y quita el **X-Powered-By.** Con un escaneo nuevo el reporte ya no tiene alertas bajas:



ZAP by Checkmarx **reporte1-zap.html**

Sitio: http://127.0.0.1:4000

Generado a mar, 10 nov 2026 17:07:19

ZAP Versión: 2.17.0

ZAP by Checkmarx

Sumario de Alertas

Nivel de riesgo	Número de Alertas
Alto	0
Medio	0
Bajo	0
Informativo	1

Alertas

Nombre	Nivel de riesgo	Número de Inst
Petición de Autenticación Identificada	Informativo	1

Politica IAM restrictiva en AWS

Para limitar el acceso al bucket solo a los roles necesarios apliqué el principio de mínimo privilegio con 3 archivos JSON, que se encuentran en la carpeta **“iam”** del repositorio:

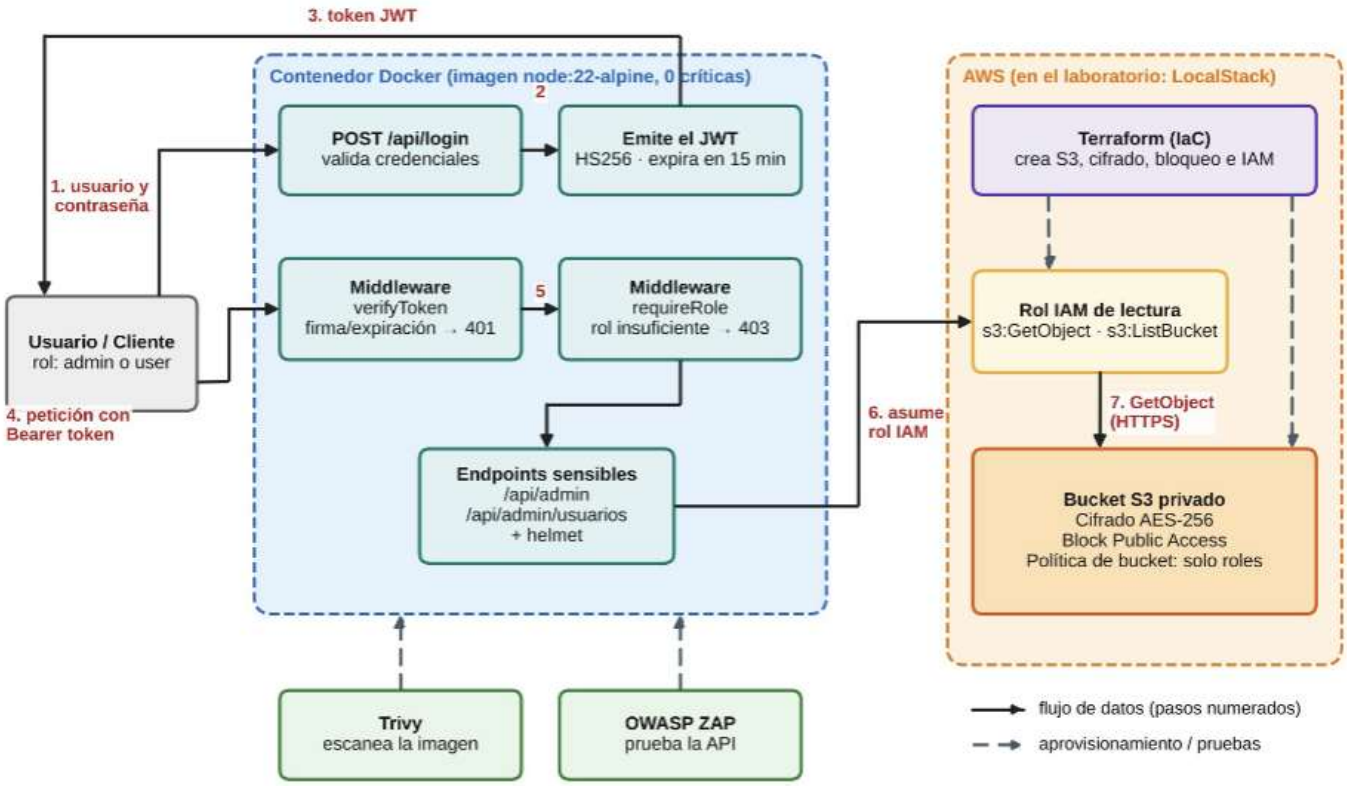
Archivo	Qué define
Política-iam-lectura.json	Permisos del rol rol-lectura-actividad3: solo s3:GetObject sobre los objetos del bucket y s3:ListBucket sobre el bucket.
Trust-policy-rol-lectura.json	Política de confianza: solo las tareas de contenedores de ECS pueden asumir el rol.
Politica-bucket.json	Política del bucket: niega todo acceso que no venga del rol de lectura o del de administración y niega petición que no use HTTPS.

Debido a que una denegación explícita tiene prioridad sobre un permiso, ni un administrador de la cuenta podría leer el bucket sin usar uno de esos roles, dichas políticas no se hicieron cumplir en el laboratorio, debido a que el plan Hobby de LocalStack no incluye la aplicación depolíticas IAM, solo valide que los JSON tuvieran formato correcto y el número de cuenta que aparece en ellos es el que usa LocalStack. En la siguiente tabla se muestra el ejemplo de uso en AWS real:

Quién	Acción	Resultado esperado
Rol de lectura	GetObject sobre prueba.txt	Permitido
Rol de lectura	PutObject	AccessDenied: no está en su política
Rol de lectura	GetObject por HTTP sin cifrar	AccesDenied por la política del bucket
Otro usuario o rol de la cuenta	Cualquier acción sobre el bucket	Denegado
Acceso anónimo	GetObject	

Diagrama de arquitectura

En la siguiente imagen se muestra la arquitectura segura que integra las 3 fases y la política IAM, debido a quela API del laboratorio es simulada y no se conecta realmente al S3, el diagrama representa la arquitectura objetivo:



El flujo de datos es el siguiente:

1. El usuario envía su usuario y contraseña a **/api/login.**
2. La API, en un contenedor Docker con node 22 alpine valida las credenciales.
3. La API emite un JWT que expira en 15 minutos.
4. El usuario envía el token como Bearer a los endpoints sensibles.
5. verifyToken valida la firma y la expiración y requireRole valida el rol.
6. La tarea de la API asume el rol IAM de lectura.
7. El rol lee el bucket por HTTPS: cifrado AES-256, sin acceso público y con una política que solo permite a los roles definidos.

Conclusión

En conclusión, realizar esta actividad me ayudó para que me quedara claro que la seguridad aplica por capas y que cada una hay que comprobarla, en la infraestructura declaré con Terraform un bucket cifrado y sin acceso público y lo consulte con AWS CLI, en el contenedor escaneé la imagen con Trivy hasta dejarla sin críticas, por último en la aplicación potegí los endpoints con JWT y roles y los probé con curl y ZAP; y e el acceso limité el bucket a roles con permisos mínimos con políticas IAM.

Al igual, varios resultados tuve que leerlos con cuidado y no darlo por hecho, tal como que LocalStack me dejó descargar un archivo sn firmar, node 18 aumentó las críticas en lugar de bajarlas y ZAP escaneó sin un token.